



IT ADMIN / OPERATOR GUIDE

pktFlow Admin Guide

Complete technical documentation for deploying, configuring, and operating the pktFlow network flow analytics platform.

CONTENTS

- | | |
|----------------------------|------------------------------|
| 1. System Requirements | 8. ClickHouse Administration |
| 2. Installation | 9. User Management |
| 3. Configuration Reference | 10. Alert Rules |
| 4. Collector Node Setup | 11. API Reference |
| 5. Ingest Pipeline Details | 12. Troubleshooting |
| 6. Verifying Ingest | 13. Backup & Recovery |
| 7. Service Management | |

1. System Requirements

Application Server

Component	Minimum	Recommended
	Linux (AL2, Ubuntu 22+, RHEL 8+)	Amazon Linux 2023
CPU	2 vCPU	4+ vCPU
	4 GB	16 GB (ClickHouse benefits greatly)
Disk	50 GB	500 GB+ SSD
	3.11+	3.12

Collector Node(s)

Linux, UDP port 2055 open from routers, HTTP reach to pktFlow API port, goflow2 + Vector installed.

Network Prerequisites

- Routers/switches must support NetFlow v9 export
- NetFlow export target: collector node IP, UDP port 2055
- Collector must reach pktFlow server API port via HTTP
- Users need browser access to pktFlow server's web port

2. Installation

```
# Clone and set up Python environment
git clone <repository-url> /opt/pktflow
cd /opt/pktflow
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

# Install ClickHouse
curl https://packages.clickhouse.com/rpm/clickhouse.repo \
  -o /etc/yum.repos.d/clickhouse.repo
yum install -y clickhouse-server clickhouse-client
systemctl enable --now clickhouse-server

# Configure pktFlow
cp config.example.yaml config.yaml
nano config.yaml # Set port, auth keys, ingest token

# Install systemd service and start
cp pktflow.service /etc/systemd/system/
systemctl daemon-reload && systemctl enable --now pktflow
```

🚨 **Security:** Always change `auth.secret_key` and `ingest.token` from their defaults before any production use.

3. Configuration Reference (config.yaml)

```
server:
  host: "0.0.0.0"
  port: 8080
  workers: 1

clickhouse:
  host: "localhost"
  port: 9000
  database: "pktflow"
  user: "default"
  password: ""

auth:
  secret_key: "CHANGE_ME_STRONG_RANDOM_32CHARS"
  algorithm: "HS256"
  token_expire_minutes: 480

ingest:
  token: "CHANGE_ME_INGEST_TOKEN"
  buffer_size: 1000
  flush_interval: 5
```

4. Collector Node Setup

Each collector runs **goflow2** (decodes NetFlow binary to JSON) piped to **Vector** (enriches and ships to pktFlow).

Vector Configuration (vector.toml excerpt)

```

[sources.netflow_in]
type = "stdin"

[transforms.parse_flow]
type = "remap"
inputs = ["netflow_in"]
source = '. = parse_json!(string!(.message))'

[transforms.enrich]
type = "remap"
inputs = ["parse_flow"]
source = '.site = if .sampler_address == "ROUTER_IP_A" { "site-a" } else { "site-b" }'

[sinks.pktflow]
type = "http"
inputs = ["enrich"]
uri = "http://<SERVER>:<PORT>/api/ingest/flows"
method = "post"
encoding.codec = "json"
batch.max_events = 500
[sinks.pktflow.auth]
strategy = "bearer"
token = "<YOUR_INGEST_TOKEN>"

```

i If systemd sets stdin to /dev/null, use Vector's `exec` source type instead of `stdin`.

5. Field Transformation Reference

Raw goflow2	After Transform	Notes
SrcAddr	src_addr	snake_case
Proto (int)	proto (string)	"TCP", "UDP", "ICMP"
NextHop: "0.0.0.0"	next_hop: ""	Empty string, not 0.0.0.0
(none)	site	Added by Vector enrich transform

6. Verifying Ingest

1

Check collector service

`systemctl status <collector-service>` — should be active (running)

2

Check pktFlow service

`systemctl status pktflow` — should be active (running)

3

Query ingest stats

`curl http://localhost:<PORT>/api/ingest/stats` — total_received should be non-zero and incrementing

7. Service Management

```
systemctl status pktflow
systemctl restart pktflow
journalctl -u pktflow -f      # live logs
journalctl -u pktflow -n 100  # last 100 lines
```

8. ClickHouse Administration

```
-- Connect
clickhouse-client --host localhost --database pktflow

-- Check table size
SELECT table, formatReadableSize(sum(bytes)), sum(rows)
FROM system.parts WHERE database='pktflow' GROUP BY table;

-- Set 90-day TTL for data retention
ALTER TABLE flows MODIFY TTL
    toDateTime(time_received_ns/1000000000) + INTERVAL 90
DAY;
```

9. User Management

Manage users from the **Users** page in the dashboard (admin login required).

Roles: admin, analyst, viewer.

Emergency CLI reset: Use Python + sqlite3 + bcrypt to set a new hashed password directly in pktflow.db, and ensure is_active=1 on the target account.

10. Alert Rule Status

Rule Type	Status	Description
	✓ Functional	No flows received from site within N seconds
new_host	✓ Functional	Previously unseen IP appears in flow data
	⚠ Placeholder	Engine stub only — does not execute
rate_spike	⚠ Placeholder	Engine stub only — does not execute
	⚠ Placeholder	Engine stub only — does not execute

11. API Reference

Method	Endpoint	Description
POST	/api/auth/login	Obtain JWT. Body: {username, password}
POST	/api/ingest/flows	Ingest flow batch. Bearer: ingest token. Body: JSON array.
GET	/api/ingest/stats	Ingest counters: total_received, total_flushed, last_flush
GET	/api/flows/query	Query flows. Params: src_ip, dst_ip, proto, site, start, end
GET	/api/flows/top-talkers	Top N source IPs by bytes in time range
GET	/api/alerts	List alert events
POST	/api/users	Create user (admin only)

12. Troubleshooting

Symptom	Fix
	Check /api/ingest/stats. If 0, check collector + Vector logs.
total_received stays 0 while access log shows POSTs	Normalizer dropping records — confirm Vector sends snake_case fields and proto as string.
	Check SQLite users table — is_active may be 0. Set to 1.
Service crashes on start	journalctl -u pktflow -n 50. Verify ClickHouse is running.

Normal — templates re-learned within seconds from next router template packet.

13. Backup & Recovery

Back up: config.yaml, pktflow.db (SQLite), pktflow.service, and ClickHouse data directory.

Restore: Stop the service → restore config.yaml and pktflow.db → restore ClickHouse if needed → start service → verify /api/ingest/stats and dashboard.